

Hybrid Intelligent Movie Recommendation System

Project Final Report

1. Abstract

This project outlines the development of a high-performance, hybrid movie recommendation system. The objective was to solve the limitations of isolated recommendation domains (e.g., exclusively Hollywood or exclusively Bollywood) and the "Cold Start" problem inherent in pure Collaborative Filtering. By synthesizing the 32 Million row MovieLens dataset with deep NLP metadata from TMDB, IMDb, and specialized regional datasets, the system successfully bridges the gap between diverse cinematic domains. The final deliverable features an Alternating Least Squares (ALS) predictive model, a 5,000-dimensional TF-IDF vector space, and an interactive Streamlit web application.

2. Dataset Integration & Methodology

2.1 The Core Datasets

- * **MovieLens 32M:** Provided the foundation for collaborative filtering, supplying 32 million behavioral rating points.
- * **TMDB & IMDb Non-Commercial:** Supplied rich metadata including genres, plot overviews, and cast/crew lists.
- * **Bollywood Regional Dataset:** Injected over 1,600 Hindi movies to ensure the model could seamlessly transition across geographical and linguistic boundaries.

2.2 Data Engineering & "The Golden Bridge"

The primary technical challenge in the integration phase was standardizing the disparate identification systems used by the various databases. The project utilized the MovieLens `links.csv` file as a "Golden Bridge", permitting horizontal `.merge()` operations across Pandas dataframes utilizing `imdbId` and `tmdbId` mapping.

For regional datasets lacking global IDs, synthetic IDs were generated, and vertical `.concat()` appends were utilized. To ensure data integrity, deduplication was dynamically handled during index generation.

3. Machine Learning Algorithms

3.1 Collaborative Filtering (PySpark)

Due to the sheer scale of the 32M dataset, distributed computing using Apache Spark (PySpark) was necessary. An **Alternating Least Squares (ALS)** matrix factorization algorithm was trained to learn latent user preferences. The model achieved a highly competitive Root Mean Squared Error (RMSE) of ~ 0.81 , indicating strong predictive accuracy for user ratings.

3.2 Content-Based Filtering (NLP)

To address the Cold Start problem, a robust Natural Language Processing (NLP) pipeline was developed:

1. **Tag Synthesis:** The `overview`, `genres`, `cast`, and `director` fields were unified into a single text corpus per movie.
2. **Cleaning & Stemming:** Regular expressions removed non-alphanumeric noise, while the NLTK `PorterStemmer` standardized vocabulary to root forms.
3. **TF-IDF Vectorization:** A `scikit-learn` pipeline transformed the text into a 5,000-feature sparse matrix, prioritizing unique, distinguishing keywords while ignoring generic terms.

3.3 The Hybrid Recommendation Engine

Instead of pre-computing an $89K \times 89K$ cosine similarity matrix—which exceeds the memory bounds of standard hardware—the system computes mathematical similarity *on-the-fly* during inference.

The final ranking is determined by a hybrid weighting function:

$\text{Score} = (\text{Cosine Similarity} \times 0.8) + (\text{Normalized IMDb Rating} \times 0.2)$

This guarantees that recommendations are not only contextually identical but also highly rated by global audiences.

4. User Interface & Error Handling

4.1 Modern Frontend

Built using Streamlit with custom CSS to implement Glassmorphism design, responsive movie cards, and a vibrant Hero Section to ensure a premium user experience.

4.2 API Resilience & Proxy Integration

Due to regional ISP blocking of major movie database APIs (TMDB/OMDB), the system was engineered with a multi-layered, fault-tolerant architecture:

1. ****Frontend Metahub CDN Routing****: The Python backend was completely bypassed for image fetching. Instead, the IMDb ID is formatted and injected directly into the HTML to utilize the Stremio Metahub Community CDN, allowing the user's browser to securely fetch posters without rate limits or API keys.
2. ****JavaScript On-Error Hot-Swapping****: If the CDN fails or the movie lacks an ID, an inline `onerror` JavaScript attribute instantly hot-swaps the broken image with a locally encoded Base64 SVG placeholder, preventing UI breakage.
3. ****Outbound IMDb Integrations****: Dynamic links were generated per-movie to allow users to click seamlessly through to the official IMDb page for deep-dive reading.

5. Conclusion

The Hybrid Intelligent Movie Recommendation System successfully demonstrates the end-to-end Machine Learning pipeline. From Big Data aggregation in PySpark and advanced NLP feature engineering in Pandas, to real-time vector math inference and resilient frontend web development, the project fulfills all objectives and provides a scalable foundation for future enhancements like Neural Collaborative Filtering.